

PROJECT REPORT

Paisa Expense Tracker

A Flask-based expense management application for recording expenses, organizing categories, managing budgets, and viewing financial summaries.

Technology: Python, Flask, SQLAlchemy, SQLite, HTML, CSS, and JavaScript

Repository: ExpenseTracker

1. Introduction

Paisha Expense Tracker is a web application designed to make personal expense management simple and organized. Users can create an account, record daily expenses, assign expenses to categories, choose a payment method, and review their spending from a personal dashboard. The application also supports budget records and financial reports so users can understand their spending over time.

The project is built with Python and Flask using a maintainable application structure. Flask handles routing and request processing, Flask-SQLAlchemy maps application data to a local SQLite database, Flask-Login protects user-specific pages, and Flask-WTF provides form protection. The interface uses server-rendered Jinja templates with custom HTML, CSS, and JavaScript.

Project Goals

- Provide a secure, user-specific place to manage expenses.
- Keep data local through an embedded SQLite database.
- Offer a clean and responsive dashboard experience.
- Use a modular Flask structure that can be extended later.

Main Features

Authentication

Registration, login, logout, and password updates.

Expense Management

Add, edit, filter, and delete personal expenses.

Categories

Create and manage user-owned expense categories.

Reports

Review monthly and category-wise spending summaries.

2. Project Structure

The project keeps application logic inside the `app` package and stores local runtime data inside `instance`.

```
ExpenseTracker/
|-- app/
|   |-- __init__.py
|   |-- auth.py
|   |-- extensions.py
|   |-- main.py
|   |-- models.py
|   |-- schema.py
|   |-- seed.py
|   |-- static/
|   |   |-- css/
|   |   |   |-- app.css
|   |   |-- js/
|   |   |   |-- controls.js
|   |   |-- favicon.svg
|   |-- templates/
|   |   |-- auth/
|   |   |   |-- login.html
|   |   |   |-- register.html
|   |   |-- errors/
|   |   |   |-- 404.html
|   |   |   |-- 500.html
|   |   |-- base.html
|   |   |-- dashboard.html
|   |   |-- expenses.html
|   |   |-- expense_form.html
|   |   |-- categories.html
|   |   |-- category_form.html
|   |   |-- reports.html
|   |   |-- account.html
|-- instance/
|   |-- expense.db
|-- config.py
|-- requirements.txt
|-- run.py
|-- .gitignore
|-- README.md
|-- expense_tracker_font_palette_mockup.html
```

Responsibilities

run.py

Application entry point used to start the development server.

config.py

Loads environment variables and configures SQLite and Flask settings.

app/__init__.py

Creates the Flask app, initializes extensions,

app/models.py

Defines User, Category, Expense, and Budget database models.

creates tables, seeds categories, and registers blueprints.

app/auth.py

Provides registration, login, and logout routes.

app/main.py

Provides dashboard, expense, category, account, and report routes.

3. Main Codes

The following excerpts show how the application starts, configures the database, and connects the backend to the interface.

run.py - application entry point

Application Entry Point: run.py

```
from app import create_app

app = create_app()

if __name__ == "__main__":
    app.run(debug=True)
```

app/__init__.py - application factory

```
def create_app(config_class=Config):
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object(config_class)

    db.init_app(app)
    migrate.init_app(app, db)
    login_manager.init_app(app)
    csrf.init_app(app)

    with app.app_context():
        db.create_all()
        ensure_category_schema()
        seed_defaults()

    app.register_blueprint(auth_bp)
    app.register_blueprint(main_bp)
    return app
```

config.py - SQLite configuration

```
PROJECT_DIR = Path(__file__).resolve().parent
INSTANCE_DIR = PROJECT_DIR / "instance"

class Config:
    SECRET_KEY = os.getenv("SECRET_KEY", "change-this-local-secret")
    SQLALCHEMY_DATABASE_URI = os.getenv(
        "DATABASE_URL", f"sqlite:/// {INSTANCE_DIR} / 'expense.db'"
    )
```

3. Main Codes Continued

app/models.py - database model

```
class Expense(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(140), nullable=False)
    amount = db.Column(db.Numeric(12, 2), nullable=False)
    expense_date = db.Column(db.Date, default=date.today, nullable=False)
    payment_method = db.Column(db.String(40), nullable=False)
    category_id = db.Column(db.Integer, db.ForeignKey("category.id"), nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False)
```

app/auth.py - protected login route

```
@auth_bp.route("/login", methods=["GET", "POST"])
def login():
    email = request.form.get("email", "").strip().lower()
    password = request.form.get("password", "")
    user = db.session.scalar(select(User).where(User.email == email))
    if user and user.check_password(password):
        login_user(user)
        return redirect(url_for("main.dashboard"))
    flash("Invalid email or password.", "error")
    return render_template("auth/login.html")
```

app/main.py - dashboard route

```
@main_bp.get("/dashboard")
@login_required
def dashboard():
    today = date.today()
    month_start = today.replace(day=1)
    expenses = db.session.scalars(
        select(Expense).where(Expense.user_id == current_user.id)
    ).all()
    total = sum((item.amount for item in expenses), Decimal("0"))
    return render_template("dashboard.html", total=total)
```

3. Frontend HTML Codes

The templates extend a shared layout and use Jinja expressions to display data returned by Flask routes.

app/templates/base.html - shared layout

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}Paisa{% endblock %}</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/app.css') }}">
</head>
<body>
  {% block content %}{% endblock %}
  <script src="{{ url_for('static', filename='js/controls.js') }}"></script>
</body>
</html>
```

app/templates/dashboard.html - dashboard output

```
{% extends "base.html" %}
{% block content %}
<main class="dashboard">
  <h1>Monthly overview</h1>
  <section class="summary-card">
    <span>Total spent</span>
    <strong>{{ format_currency(total) }}</strong>
  </section>
  {% for expense in recent %}
    <div>{{ expense.title }} - {{ format_currency(expense.amount) }}</div>
  {% endfor %}
</main>
{% endblock %}
```

app/templates/expense_form.html - expense form

```
<form method="post">
  <label for="title">Expense title</label>
  <input id="title" name="title" required>
  <label for="amount">Amount</label>
  <input id="amount" name="amount" type="number" step="0.01" required>
  <button type="submit">Save expense</button>
</form>
```

4. Output: Run and View in Browser

1 Open PowerShell or a terminal in the project directory.

2 Install the required Python packages:

```
python -m pip install -r requirements.txt
```

3 Start the Flask application:

```
python run.py
```

4 Open the following address in a browser: **http://127.0.0.1:5000**

5 Register a user account, log in, and use the dashboard to add expenses and view reports.

Expected Browser Output

The browser opens the Paisa Expense Tracker interface. A new visitor is directed to the login page. After registration and login, the dashboard shows the current month's spending, budget information, recent expenses, and navigation to expenses, categories, reports, and account settings.

Database Output

On the first run, the application creates the local database at `instance/expense.db`. Default categories are seeded automatically. The database remains local and is not connected to an external database service.